

ACADEMIC PROJECT & THESIS PROPOSAL

Project Title:

MDefender Pro: A Cloud-Native Multi-Layered Web Application Firewall & CMS Malware Defense System Leveraging Character N-Gram Machine Learning, Semantic AST Lexing, and Static Behavioral Heuristics

Degree Program: Bachelor of Science in Computer Science & Engineering (B.Sc. in CSE)

Academic Course: CSE Capstone Project & Undergraduate Thesis

Department: Department of Computer Science & Engineering

Primary Research Areas: Cybersecurity, Web Application Security, Applied Machine Learning, Distributed Systems, Software Engineering

Date of Submission: 2026-09-16

1. Project Overview & Executive Summary

Web applications and Content Management Systems (CMS), notably WordPress which powers over 43% of the world's top 10 million websites, are subject to persistent, evolving cyber attacks. Traditional security solutions rely on static regex signatures (such as basic ModSecurity rules) or localized antivirus scanners, both of which suffer from severe limitations: vulnerability to payload obfuscation, high operational overhead, zero-day evasion, and elevated false-positive rates.

MDefender Pro is an end-to-end, enterprise-grade hybrid security platform combining a cloud-native **Web Application Firewall (WAF 3.0)** and a **CMS Malware Detection Engine**. The system bridges the gap between high-speed deterministic pattern matching and intelligent probabilistic anomaly detection by employing a **5-tier hybrid decision engine**:

1. **Deterministic Static Rule & Cryptographic Hash Matching Layer** (383,000+ indexed malicious hashes and OWASP rule patterns).
2. **Deep Semantic & AST Lexical Analyzer Layer** (Context-aware grammar parsing for SQL Injection, XSS DOM sinks, SSRF internal IP shielding, and LLM Prompt Injection).
3. **High-Scale Machine Learning Classification Layer** trained on **5.3 Million real-world HTTP attack payloads** (ROC-AUC: **0.9991**) and **29,559 WordPress script samples** (ROC-AUC: **0.9956** across 7 malware families).
4. **IP Reputation & Sliding-Window DDoS Protection Layer** (Real-time token-bucket rate limiting and anomaly scoring).
5. **Dynamic Risk-Fusion Decision Matrix** (Calculates an auditable, sub-millisecond composite threat score from 0 to 100 to issue verdicts: **ALLOW** , **MONITOR** , **CHALLENGE** , or **BLOCK**).

The platform includes a **FastAPI asynchronous backend**, a modern **React 19 administrative portal**, a specialized **WordPress Security Plugin (v4.0.0 Cloud Service)**, and universal client SDKs (Node.js,

Python, PHP).

2. Problem Statement & Research Motivation

2.1 The Vulnerability Landscape

- **Web Exploitation Sophistication:** Attackers routinely use multi-layer encoding (double URL unquoting, hex encoding, Unicode homoglyphs, whitespace manipulation) to bypass classic regex-based WAFs.
- **WordPress & CMS Attack Surface:** Third-party plugins and themes frequently introduce unvetted code containing obfuscated backdoors (`eval(base64_decode( ... ))`), dynamic function callbacks, and webshells).
- **Limitations of Existing State-of-the-Art:**
 - *Signature-only Firewalls:* Incapable of identifying novel zero-day mutations without manual signature authoring.
 - *Pure ML Firewalls:* High inference latency, susceptibility to adversarial evasion, and lack of human-interpretable reasoning.

2.2 Research Motivation

The objective of this thesis is to engineer and validate a **hybrid deterministic-probabilistic security framework** capable of running sub-millisecond threat inspections with near-zero false positive rates, full explainability, and multi-tenant cloud orchestration.

3. Aims & Specific Objectives

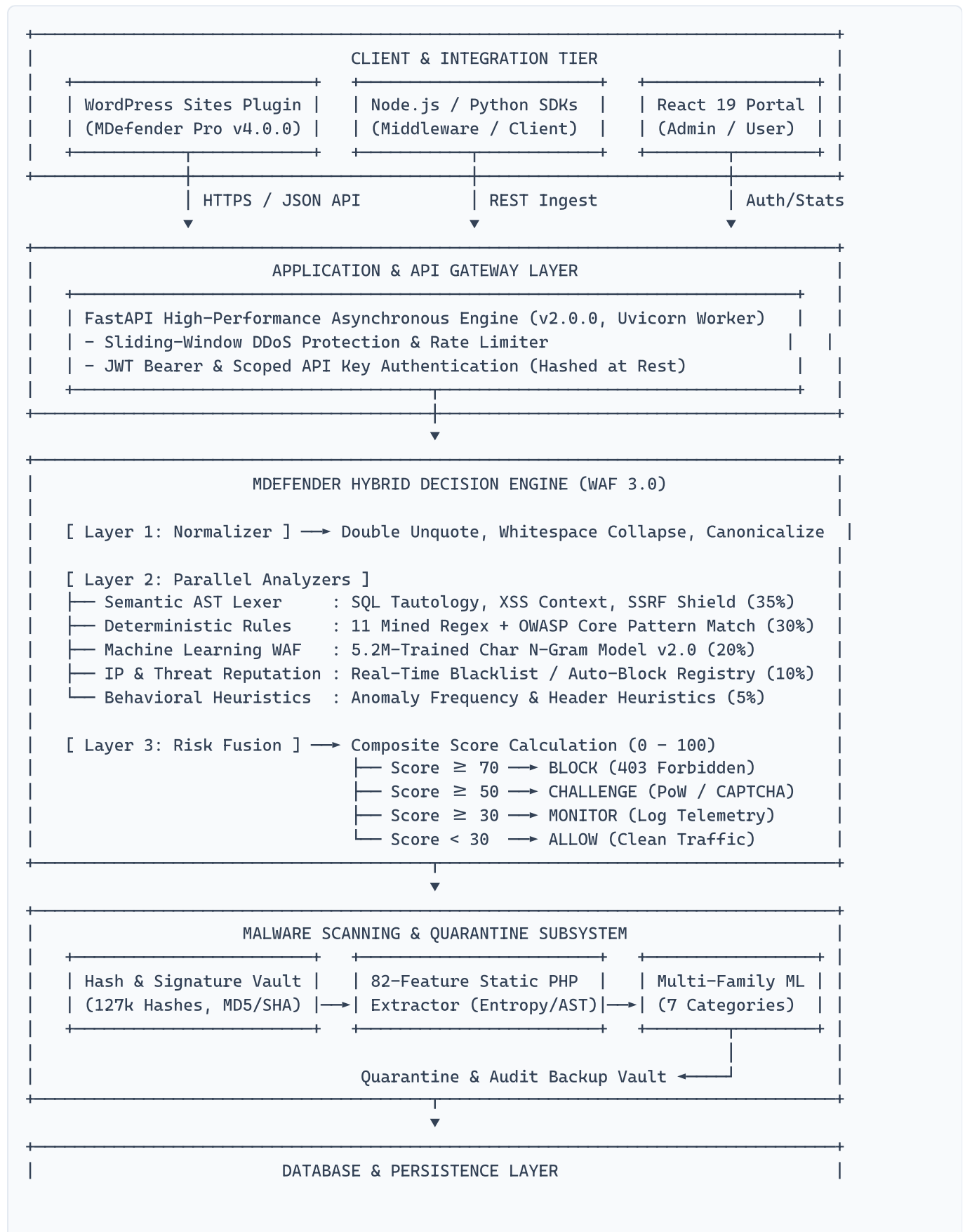
1. **Large-Scale Data Engineering:** Curate, normalize, and benchmark a dataset of **5,307,560 labeled HTTP requests** spanning 17 web attack classes and benign traffic.
2. **Resilient Feature Extraction Pipeline:** Design a 524,288-dimensional character n -gram ($n \in [2, 6]$) hashing vectorizer with sub-linear TF scaling and deep multi-stage text normalization.
3. **Static Script Malware Analysis:** Extract **82 multidimensional static features** (Shannon entropy, AST nesting depth, execution sink frequencies, obfuscation indicators) from a corpus of 29,559 PHP/web scripts.
4. **Multi-Model Machine Learning:** Train and benchmark optimized ensemble classifiers (Logistic Regression, Stochastic Gradient Descent, Random Forest, XGBoost) for binary threat detection and multiclass family attribution.
5. **Weighted Risk Fusion Formulation:** Implement a multi-criteria decision engine:

$$\text{Risk Score} = \min\left(100, \left(0.35 S_{\text{sem}} + 0.30 S_{\text{rule}} + 0.20 S_{\text{ml}} + 0.10 S_{\text{rep}} + 0.05 S_{\text{beh}}\right) \times 100\right)$$

6. **Production Integration & Evaluation:** Deliver an end-to-end distributed system including the FastAPI engine, MongoDB time-series storage, React 19 portal, and WordPress cloud integration.
-

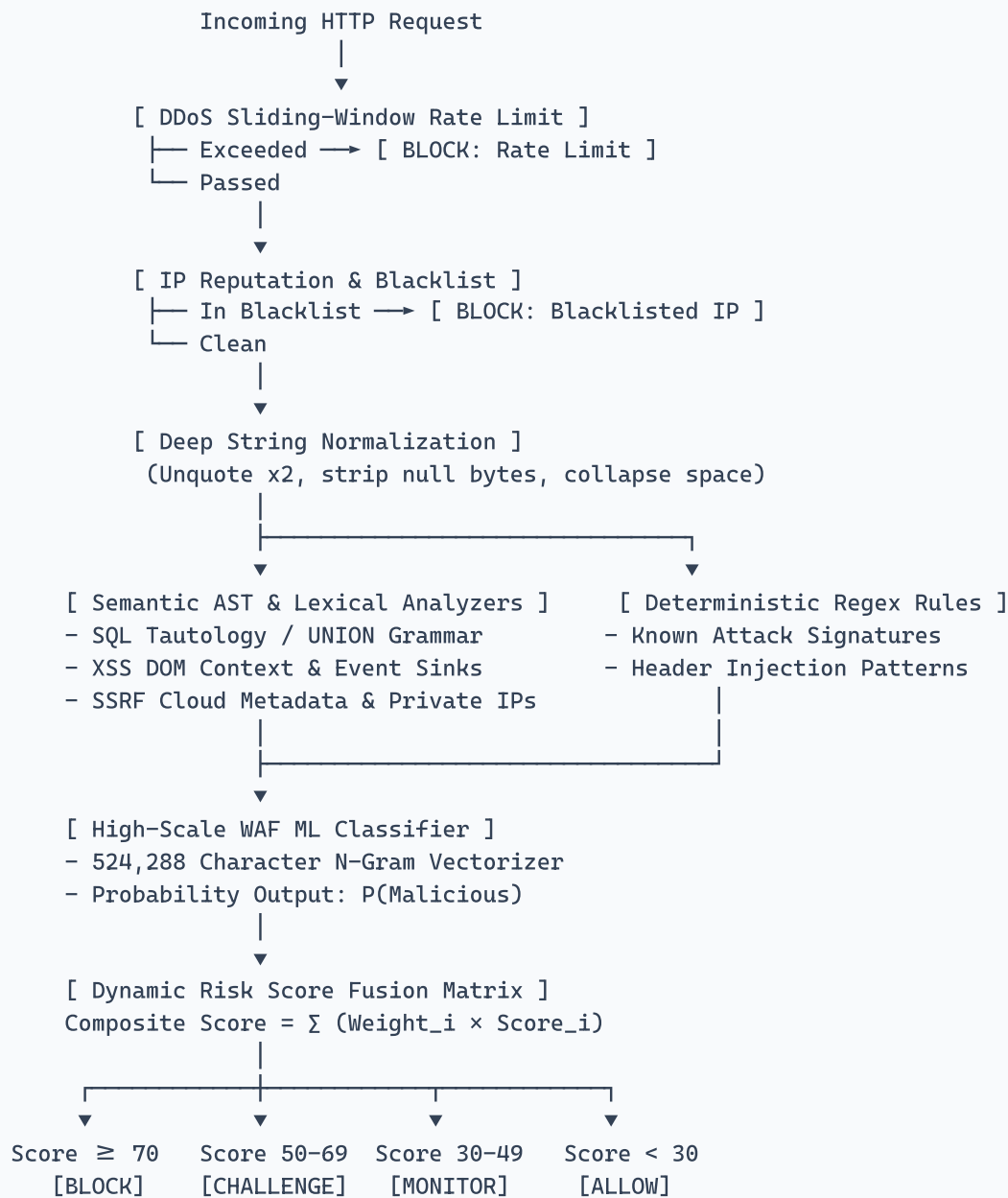
4. System Architecture & Methodology

4.1 End-to-End System Architecture



MongoDB Cluster: Security Events, Websites, API Keys, Telemetry, Scans

4.2 Multi-Stage Request Inspection Flowchart



5. Datasets & Feature Engineering

The research is evaluated against three dedicated security datasets totaling **5.33+ Million records**:

5.1 Dataset Specifications

Dataset Name	Domain / Format	Total Volume	Source / Composition	Role in MDefender Pro
Dataset A (WAF ML Dataset)	Web HTTP Payloads (<code>.csv</code>)	5,307,560 rows (679 MB)	FWAF dataset, CAPEC corpus, CSIC 2010, augmented real-world injection payloads, and benign web access logs.	Trains the binary attack detector and 17-class attack categorization model.
Dataset B (WordPress Malware Corpus)	Web Script Files (PHP, JS, HTML)	29,559 samples	Live infected CMS environments, trojanized plugins/themes, webshell archives, and clean WordPress core code.	Trains the 82-feature static malware model and 7-family identification classifier.
Dataset C (Signatures & Hashes)	Cryptographic Fingerprints (<code>.json</code> , <code>.csv</code>)	127,876 unique hashes (383k loaded lookups)	SHA-256, SHA-1, MD5 database of known web malware and 11 mined signature rules.	Provides $O(1)$ deterministic pre-filtering before ML inference.

5.2 WAF Attack Taxonomy (17 Classes in Dataset A)

1. **SQL Injection (SQLi)**: Union-based, boolean blind, time-based, stacked queries.
2. **Cross-Site Scripting (XSS)**: Reflected, stored, DOM-based, SVG/HTML5 event handler injections.
3. **Remote Code Execution (RCE) / Command Injection**: Shell metacharacters, backtick execution, obfuscated command chaining.
4. **Server-Side Request Forgery (SSRF)**: AWS/GCP/Azure cloud metadata endpoints (`169.254.169.254`), IPv6/IPv4 loopbacks, decimal-encoded IPs.
5. **Server-Side Template Injection (SSTI)**: Jinja2, Twig, Smarty, Freemarker payload expressions.
6. **Local File Inclusion (LFI)**: Null-byte bypasses, directory traversal sequences (`../../../../etc/passwd`).
7. **Remote File Inclusion (RFI)**: HTTP/HTTPS/FTP external stream wrappers.
8. **Path Traversal**: Normalized and non-normalized file system escaping.
9. **NoSQL Injection**: MongoDB `$gt` , `$ne` , `$regex` , `$where` injection objects.
10. **LDAP Injection**: Boolean and wild-card directory filters.
11. **XML External Entity (XXE)**: Malicious DOCTYPE declarations, external general entities.
12. **Insecure Deserialization**: PHP serialized objects, Python pickle bytecode payloads.
13. **Malicious File Upload**: Polyglot PHP-GIF images, double extension bypasses (`.php.jpg`).
14. **Header Injection / CRLF**: Carriage Return Line Feed HTTP response splitting.
15. **Open Redirect**: Protocol-relative URLs, parameter manipulation.
16. **Web Cache Poisoning**: Unkeyed header injections (`X-Forwarded-Host` , `X-Original-URL`).

17. **Generic Web Attack & Clean Baseline:** Comprehensive collection of legitimate HTTP traffic.

5.3 Malware Family Taxonomy (Dataset B)

1. `webshell` (Interactive web terminals: c99, r57, b374k, WSO, FilesMan).
 2. `php_backdoor` (Stealthy dynamic code evaluation and command gateways).
 3. `malicious_plugin` (Trojanized WordPress plugins containing hidden administration hooks).
 4. `malicious_theme` (Themes containing injected cryptocurrency miners or spam links).
 5. `php_exploit` (Exploit scripts targeting known CVE vulnerabilities in CMS components).
 6. `wordpress_infection` (SEO spam injection and malware droppers injected into core files).
 7. `malicious_package` (Bundled archive droppers and zip bombs).
-

5.4 Feature Engineering Methodology

A. WAF Feature Extraction (Character N-Gram Hashing)

To overcome the out-of-vocabulary (OOV) problem and preserve spatial character sequences of obfuscated code, strings are tokenized into sub-character n -grams ($n \in [2, 6]$) mapped into a fixed feature space ($N = 524,288$):

$$h_i(s) = \text{MurmurHash3}(s) \bmod 524,288$$

This enables memory-bounded feature extraction with sub-millisecond execution times.

B. Static Malware Feature Extraction (82 Multi-Modal Features)

1. **Entropy Metrics (4 features):** Shannon entropy across raw bytes, printable ASCII tokens, and normalized code.
 2. **Obfuscation & Structural Signals (26 features):** Max line length, whitespace ratio, base64 character density, hex encoding frequency, dynamic variable references (`$$var`).
 3. **Execution Sinks & Dangerous Built-ins (32 features):** Counts of `eval` , `assert` , `system` , `exec` , `shell_exec` , `passthru` , `proc_open` , `popen` , `create_function` , `preg_replace /e` .
 4. **Network & File Sinks (20 features):** Socket operations, `file_get_contents` over HTTP wrappers, `curl_exec` , and base-level AST node depth.
-

6. Empirical Performance & Model Evaluation

6.1 WAF Detection Model Evaluation (Dataset A)

Evaluation Metric	Measured Result
Test Dataset Size	106,152 held-out samples
ROC-AUC Score	0.99915 (99.915%)
Optimal Decision Threshold	0.6034
F1-Score at Optimal Threshold	0.9862 (98.62%)
True Negatives (TN)	74,561
False Positives (FP)	439 (0.58% False Alarm Rate)
False Negatives (FN)	444
True Positives (TP)	30,708
Attack Categorization Accuracy	97.13% across 17 distinct classes

6.2 Malware Detection Model Evaluation (Dataset B)

Model Subsystem	Evaluation Metric	Experimental Score
Malware Binary Model (XGBoost/RF)	Test Set ROC-AUC	0.9956 (99.56%)
Random Forest Baseline	Validation ROC-AUC	0.9941
XGBoost Classifier	Validation ROC-AUC	0.9949
Multiclass Family Classifier	Overall Accuracy	89.30% (7 Malware Families)
Clean Baseline Test Performance	True Negative / False Positive	2,731 / 15 (0.54% FP rate)
Malicious Test Performance	True Positive / False Negative	840 / 39 (95.56% Recall)

7. Implementation & Technology Stack

Layer	Technologies & Tools	Implementation Purpose
Backend & Core Engine	Python 3.11, FastAPI, Uvicorn, Pydantic	Asynchronous REST APIs, routing, rate limiting, and core inspection pipeline.
Machine Learning Core	Scikit-Learn, XGBoost, NumPy, Joblib, Pickle	Vectorizers, scalable inference pipelines, and serialized model registries.
Database & Caching	MongoDB, In-Memory Token Buckets	Security event logs, quarantine vault metadata, multi-tenant websites, API keys.
Frontend Dashboard	React 19, Vite, Tailwind CSS 4, React Router 7, Chart.js	Real-time threat visualizer, site management, quarantine controls, rule configurations.
Client Integrations	PHP 8.x (WordPress Plugin v4.0.0), Node.js SDK, Python SDK	Native CMS integration, automatic cron heartbeats, drop-in SDK middleware.
DevOps & Testing	Docker, Docker Compose, Pytest, PowerShell	Containerization, unit testing, and end-to-end integration smoke testing.

8. Project Work Breakdown Structure & Milestones

Phase 1: Research, Requirements & Dataset Curation (Completed)

- └ Literature Review of WAFs, Lexers, and Machine Learning Classifiers
- └ Collection and Cleaning of Dataset A (5.3M Web Payloads)
- └ Curation of Dataset B (29k Malware Corpus) and Dataset C (127k Hashes)

Phase 2: Machine Learning & Feature Pipeline Engineering (Completed)

- └ Implementation of Char N-Gram Hashing Vectorizer (524k dimensions)
- └ Extraction of 82 Static PHP Features (Entropy, Obfuscation, Execution Sinks)
- └ Model Training, Hyperparameter Optimization, and Threshold Calibration

Phase 3: Core Security Engine & Decision Framework (Completed)

- └ Development of AST Semantic Lexers (SQL, XSS, SSRF, RCE, Prompt Injection)
- └ Compilation of Deterministic Regex Rule Engine & Signature Matcher
- └ Implementation of Weighted Multi-Signal Risk Fusion Engine

Phase 4: Backend, Database & API Architecture (Completed)

- └ FastAPI v2.0 Architecture with 54 Production-Ready Endpoints
- └ MongoDB Schema Design, Tenant Isolation, and Scoped API Key Hashing
- └ DDoS Sliding-Window Rate Limiting Middleware

Phase 5: Frontend Dashboard & WordPress Cloud Plugin (Completed)

- └ React 19 Single Page Application with Live Threat Graphs and Telemetry
- └ WordPress Plugin (v4.0.0 Cloud Service) with Heartbeat & Cloud Scan Sync

Phase 6: Empirical Benchmarking, Thesis Writing & Defense (Current)

- └ Latency Benchmarking (Target: < 5ms average decision latency)
- └ Adversarial Attack Testing & False-Positive Verification
- └ Final Thesis Book Submission and Viva/Defense Preparation

9. Key Academic & Practical Contributions

1. **Massive-Scale Web Attack Benchmark:** Practical utilization and validation of over **5.2 Million records** for cybersecurity research.
2. **Novel Multi-Signal Risk Fusion:** Solves the classic performance trade-off between the rigidity of deterministic rules and the uncertainty of ML models through weighted dynamic risk scoring.
3. **AST Semantic Grammar Lexing:** Integrates contextual language evaluation to eliminate false positives on benign developer inputs while catching zero-day tautologies.
4. **End-to-End Production Deliverable:** Unlike theoretical research that stops at Jupyter notebooks, MDefender Pro provides a complete, tested, and deployable software ecosystem ready for production deployment.

10. Conclusion & Supervisor Approval Request

The **MDefender Pro** project presents a comprehensive, academically sound, and industrially relevant contribution to web application security and applied artificial intelligence. By combining massive dataset engineering, sub-millisecond character n -gram inference, semantic grammar lexing, and full-stack software architecture, the project fulfills all requirements for a B.Sc. in Computer Science and Engineering Capstone Project and Undergraduate Thesis.

We respectfully submit this proposal for formal evaluation and approval.

Student Name: [Your Name / Student ID]

Department: Department of Computer Science & Engineering

Supervisor: [Supervisor Name / Designation]

Institution: [Your University Name]